

5.1

Expensive overhead = # of visited nodes that are not fully reported subtrees $\rightarrow$ type 2 nodes
 $\text{Region}(v) \cap R \neq \emptyset$ but not reported as whole subtree because $\text{region}(v) \not\subset R$

Proving $O(\sqrt{n})$

Count defined as $Q(n)$ recurrence that for $n > 1$, $Q(n) = 2 + 2Q(n/4)$ With $Q(1) = O(1)$

Go down two levels $\rightarrow$ vertical line intersects only 2 of 4 grandchild regions after two levels, for +2, then 2 problems of $n/4$ size

Then solve for $Q(n) = 2 + 2Q(n/4)$

Let $n = 4^k$

Let $S(k) = Q(4^k)$, so recurrence becomes $S(k) = 2 + 2S(k-1)$ and $S(0) = Q(1) = O(1)$

$S(k) = 2 + 2S(k-1)$

$= 2 + 2(2 + 2S(k-2))$

$= 2 + 2^2 + 2^2 * S(k-2)$

After i steps, $S(k) = (2 + 2^2 + \dots + 2^i) + 2^i * S(k-i)$

Then set $i = k \rightarrow S(k) = (2 + 2^2 + \dots + 2^k) + 2^k * S(0)$

So geometric sum is $2 + 2^2 + \dots + 2^k = 2(2^k - 1)$

Then, $S(k) = 2(2^k - 1) + 2^k * S(0) = O(2^k)$

Since $n = 4^k$, then $2^k = \sqrt{4^k} = \sqrt{n}$.

Therefore $Q(n) = O(\sqrt{n})$

Proving $O(\sqrt{n})$

Prove that some set P with n points and axis-aligned rectangle R such that kd-tree built by median splits is forced to visit $\Omega(\sqrt{n})$ nodes even when output size k is small

Make $n = m^2$ points on an $m \times m$ grid

- Let $m = \sqrt{n}$ and assume n is square

$P = \{(i, j) \mid i = 1, 2, \dots, m, j = 1, 2, \dots, m\}$

- Points are spread across plane so kd-tree splits keep cutting space into rectangles

Choose a vertical rectangle between two consecutive x-columns so it contains no grid points

$R = [t, t + \varepsilon] \times [-\infty, \infty]$

- Need report points with x in tiny gap, slab cuts through lots of kd-tree regions

Algorithm must visit $\Omega(\sqrt{n})$ nodes

Let L be the left boundary line of query slab: $x = t$

Define $Q(n) = \#$ of kd-tree regions whose interior is intersected by L , among kd-tree built on n points

- Every time L intersects a node's region, but it is not fully contained in R .

- Algorithm must visit that node (cannot prune)

- Therefore number of visited type 2 nodes is $\Omega(Q(n))$

- Need to show that $Q(n) = \Omega(\sqrt{n})$

Now show what happens after two levels of a balanced 2D kd-tree

- Level 1 splits by vertical line, producing left and right child regions

- Level 2 splits child nodes by horizontal line (y-median), producing four grandchildren with each having about $n/4$ points

Choose t so L passes through the middle x-slab created by median split

- L intersects both top and bottom grandchildren on side it passes through

- Gives at least 2 grandchildren of size $= n/4$

Now same structure for recurrence, but it is a minimum

$Q(n) \geq 2 + 2Q(n/4)$, $Q(1) \geq 1$

Let $n = 4^h$

$$Q(n) \geq 2 + 2Q(n/4) \geq 2 + 2(2 + 2Q(n/16)) = 2 + 2^2 + 2^2Q(n/16)$$

After h steps:

$$Q(n) \geq (2 + 2^2 + \dots + 2^h) + 2^h Q(1)$$

Sum is $\Theta(2^h)$ and since $n = 4^h$, we have $2^h = \sqrt{n}$.

Therefore, $Q(n) = \Omega(\sqrt{n})$

Since query rectangle R is tiny slab, no kd-tree region is fully contained in R unless it lies completely inside of the slab, which does not happen. So all intersected regions correspond to unreported nodes, and algorithm visits at least $c\sqrt{n}$ for constant c , even when $k = 0$

So kd tree range search has worst case query time $\Omega(\sqrt{n} + k)$ and with this construction is $\Omega(\sqrt{n})$

5.2

Insert P into kd-tree:

1. Start at the root with depth $d = 0$
2. Let the split axis be $a = d \bmod k$
3. Compare $p[a]$ with the node's point $q[a]$
 - If $p[a] < q[a]$, then go left
 - Else go right
4. Repeat until you hit a null child, then create a new leaf node storing p there
5. Update any stored bounding boxes on path upward

Time: $O(h)$, where $h = \text{tree height}$.

Delete p from kd-tree: Need to remove node while keeping the kd-tree property

1. Search for the node v containing point p using same comparisons as insert
2. If not found, stop

If found at depth d with split axis $a = d \bmod k$:

Case A: v is a leaf

- Remove it and set parent's pointer to null

Case B: v has a right subtree

- Find a replacement point $r = \text{point in } v.\text{right} \text{ with minimum coordinate on axis } a$
- Run a find-min search in that subtree
- Copy r into v , then recursively delete r from $v.\text{right}$

Case C: No right subtree but has a left subtree

- Find $r = \text{point in } v.\text{left} \text{ with minimum coordinate on axis } a$, and copy it into v
- Then set $v.\text{right} = v.\text{left}$, set $v.\text{left} = \text{null}$, and recursively delete r from the new right subtree

Runtime: $O(h)$ for the search + $O(h)$ for each find-min/delete step $\rightarrow$ worst case $O(n)$

5.3A

Algorithm:

BuildKD(p , depth)

1. If $|P| = 0$, then return null
2. Split dimension $a = \text{depth} \% d$
3. Find median point m of P by coordinate a
4. Create a node storing m and the split dimension a
5. Recurse the left child = BuildKD($\{p \text{ in } P \setminus \{m\} : p[a] < m[a]\}$, depth+1)
6. Recurse the right child = BuildKD($\{p \text{ in } P \setminus \{m\} : p[a] \geq m[a]\}$, depth+1)
7. Then return the node

- Any tie rule works for differentiating
- Each input point is stored in exactly one kd-tree node, and each node has only constant extra fields, so Total nodes = $n \rightarrow$ storage is $\Theta(n)$
- Time = $O(n \log n)$
- At each recursive call on a set of size n' :
- Find the median by coordinate a can be done in $O(n')$ time using linear-time selection
- Partitioning into left and right child is also $O(n')$ time
- Therefore the time satisfies the $T(n) = 2T(n/2) + O(n)$
- And through the master theorem, $T(n) = O(n \log n)$

5.4A

For 2D kd-tree on n points, need partial match query like $x = x_0$ or $y = y_0$ reporting all matching

points. K is number of points reported. Need to show time $O(\sqrt{n} + k)$

Algorithm (Query $x = x_0$)

At node v storing point p_v , with split axis alternating x, y

1. Check and report $\rightarrow$ if $p_v.x = x_0$ then output p_v
2. If split axis is x :
 - If $x_0 < p_v.x$, then recurse only the left
 - If $x_0 > p_v.x$, then recurse only right
 - If $x_0 = p_v.x$, then recurse both as matches can be on either side
3. If split axis is y , then must recurse to both children since splitting by y does not tell you

which

side has $x = x_0$

- Search follows vertical line $x = x_0$, except at y -split nodes when you branch

Runtime:

Similar to the range-query analysis, split the visited nodes into:

Reporting Work: $O(k)$

Type 2 extra visited nodes: Nodes whose regions intersect the query line $x = x_0$ but can not be pruned

- Suffices to bound the number of node-regions intersected by the vertical line $x = x_0$

Vertical Line Intersects $O(\sqrt{n})$ kd tree regions:

Let $Q(n)$ = maximum number of regions in a balanced 2D kd tree on n points that fixed vertical line can intersect

Do x split then y split

- After x split, vertical query line lies in only one of the two children. If split is hit, tie changes constant
- After the next y -split, that child is split into two grandchildren and the vertical line intersects both of them

So in two levels, add constant amount of boundary crossings and recurse into 2 subproblems of size $= n/4$:

$$Q(n) \leq 2 + 2Q(n/4) \text{ with } Q(1) = O(1)$$

This equates to $Q(n) = O(\sqrt{n})$

Therefore total query is $O(\sqrt{n}) + O(1) = O(\sqrt{n} + k)$

5.4B

2D range tree is a primary BST on x and at every node b , structure $A(b)$ is the points in b 's subtree stored by sorted y

- To answer $x = x_0$, range is $[x_0, x_0] \times (-\infty, \infty)$

Query Algorithm:

1. In primary x-tree, find leaf position where x would be

2. Compute split node for the interval $[x_0, x_0]$. Find search paths to x_0 as left and right boundary

3. Collect $O(\log n)$ canonical nodes whose x-ranges together cover exactly $x = x_0$

4. For each node v , report all points in $A(v)$ with any y . Output all points in $A(v)$.

- Just need scan and output and no search, as y is unrestricted

Query Time:

- Find split and canonical nodes = $O(\log n)$

- Output: $O(k)$ total since each point is outputted once

Total query time = $O(\log n + k)$

5.4C

Balanced BST on the x-coordinate

- Build a balanced BST keyed by distinct x-values, where each BST node for key x stores a linked list / dynamic array of all points (x, y) having that same x

- Each point is stored exactly once in exactly one bucket $\rightarrow O(n)$ space

Querying:

1. Search the BST for key x_0 , as standard BBST search takes $O(\log n)$

2. If found, output all points in that node's list, which cost $O(k)$

- If not found, return empty

Total Runtime: $O(\log n + k)$

5.5A:

Need to force $\text{SearchKdTree}(v, R)$ to recurse into almost every subtree because the query triangle intersects every node region. Only prune when $\text{region}(\text{child}) \cap R = \emptyset$, and otherwise you recurse.

1. Put points on diagonal $y = x$

- Let $n = m$ points be $P = \{(4i, 4i) \mid i = 1, 2, \dots, n\}$

- Build balanced 2D kd tree on P in usual way by alternating vertical and horizontal splitting lines

- Each node corresponds to an axis-aligned rectangle $\text{region}(v)$ defined by ancestor splits

- Because all points satisfy $y = x$, every node region that contains at least one point must intersect diagonal line $y = x$

2. Choose triangle that intersects every region but contains no points

- Pick very thin slice of diagonal band above the line $y = x$, but make it a triangle

- Let R be the triangle bounded by the three lines:

- $y = x + 1$, $y = x + 2$, and $x = 4n + 10$

- This is a long triangle shaped strip between two lines parallel to $y = x$ that is closed on the right

- Every point $(4i, 4i)$ has $y - x = 0$ but every point in R has $1 \leq y - x \leq 2$.

- So $R \cap P = \emptyset$, hence $k = 0$

3. Kd tree search visits $\Omega(n)$ nodes

- Consider any node v in the kd-tree. Region is axis rectangle

- v 's subtree contains some diagonal points, so $\text{region}(v)$ must contain some segment of the diagonal $y = x$

- For axis rectangle that spans diagonal points, quantity $y - x$ ranges over interval of length at least the rectangle's width and height
- With spacing, leaf-level rectangles have enough width so strip $1 \leq y - x \leq 2$ intersects rectangle

Therefore for every node v , $\text{region}(v) \cap R \neq \emptyset$

- This means SearchKdTree cannot prune either child using the empty intersection case
- Also R is thin and contains no points, so never get $\text{region}(v) \subseteq R$, so never

ReportSubtree and stop early

Therefore the recursion keeps going down the entire tree, visiting $\Theta(n)$ nodes, even though $k = 0$

There exists a point set with all points on diagonal $y = x$ and a triangle query range R with no reported points such that kd tree search visits $\Theta(n)$ nodes. Hence triangle range queries have linear worst case query time $\Omega(n)$